Innopolis University

Big Data — Final Project

Temporal Analysis and Prediction of Job Demand Using LinkedIn Data

Team 32

Emil Goryachih — Data Engineering (Stages 1, 2)
Vladislav Galkin — EDA & Analytics (Stage 2)
Rodion Krainov — Machine Learning (Stage 3)
Vladislav Galkin — Dashboard & Visualisation (Stage 4)

Dataset:	1.3 M LinkedIn Jobs and Skills (2024)
Task type:	Binary classification (high vs. low demand)
Best model:	Gradient Boosted Trees, AUC ROC 0.94
Repository:	`bigdata-team32-linkedin-job-demand`

May 12, 2026

Contents

1 Introduction

The labour market is one of the most volatile, geographically uneven, and information-rich domains in the modern economy. Online job platforms such as LinkedIn aggregate millions of postings every week, each of which is a small signal about which roles, skills, companies and regions are growing or contracting. Aggregated across the platform, those signals describe *job demand*: the structural pattern of where, what, and at what seniority companies need to hire.

Business objective. This project builds an end-to-end big data pipeline that turns 1.35 million raw LinkedIn job postings into actionable demand intelligence. The pipeline answers two complementary business questions:

(Q1) **Descriptive:** where is hiring concentrated by country, role, seniority, work arrangement, and company? Which skills are most demanded globally and per country?

(Q2) **Predictive:** given a (country, position) combination, can we predict whether it is a *high-demand* segment for that role family?

Both outputs feed an interactive Apache Superset dashboard so that business stakeholders — recruiters, HR analysts, course designers and policy researchers — can drill into the market signal without running any Spark job themselves.

Why big data. The raw dataset (**1 348 454** rows, **6.19 GB**) is too large to process in a notebook on a single machine: the join between postings, skills and summaries by itself produces hundreds of millions of intermediate rows, and the per-position quantile computations used for the ML label require a global sort. The project therefore uses the standard university Hadoop / YARN cluster with Sqoop, Hive, Spark and Superset. All training runs on the cluster and all heavy artefacts live on HDFS.

Document structure. Section 2 describes the dataset. Section 3 gives the high-level architecture and stage I/O. Section 4 covers data preparation in PostgreSQL and Hive (ER diagram, ingestion, Parquet/enriched tables). Section 5 reports the eight EDA insights. Section 6 is the machine-learning section: feature engineering, four models with cross-validated grid search, evaluation, soft-voting ensemble and sample predictions. Section 7 documents the Superset dashboard. Section 8 concludes, and Section 9 contains challenges, recommendations and the per-task contribution table.

2 Data Description

2.1 Source

The dataset is the public Kaggle dataset *1.3M LinkedIn Jobs and Skills (2024)* by Aniczka [1]. It is composed of three CSV files joined on the natural key `job_link`:

File	Records	Size	Role
`linkedin_job_postings.csv`	1 348 454	≈ 380 MB	Posting metadata, location, level, type
`job_skills.csv`	1 296 381	≈ 850 MB	Comma-separated skill list per posting
`job_summary.csv`	1 297 332	≈ 5.0 GB	Long-form job description text
Total	—	**≈ 6.19 GB**	—

2.2 Feature description

After joining the three files on `job_link`, every row is one LinkedIn job posting with the columns listed in Table 1.

Table 1: Columns of the joined LinkedIn job-postings table.

Column	Type	Description
`job_link`	STRING	Unique LinkedIn job URL (primary key).
`last_processed_time`	TIMESTAMPTZ	Timestamp the scraper last updated the record.
`got_summary`	BOOLEAN	Whether a job summary was scraped.
`got_ner`	BOOLEAN	Whether named-entity recognition ran on the summary.
`is_being_worked`	BOOLEAN	Scraper internal flag.
`job_title`	STRING	Free-text title written by the recruiter.
`company`	STRING	Hiring company.
`job_location`	STRING	Free-text location string.
`first_seen`	DATE	Date the scraper first observed the posting.
`search_city`	STRING	City used in the search that returned this posting.
`search_country`	STRING	Country used in the search.
`search_position`	STRING	Normalised role keyword used in the search.
`job_level`	STRING	Seniority label (Entry, Associate, Mid–Senior, Director, Internship).
`job_type`	STRING	Work arrangement (Onsite, Remote, Hybrid, Full-time, Part-time, Contract).
`job_skills`	STRING	Comma-separated list of required skills.
`job_summary`	STRING	Long-form job description.

2.3 Dataset criteria compliance

The course rubric [2] requires ≥ 500 **k records**, ≥ 500 **MB**, ≥ 6 **explanatory features**, and either a *datetime* or a *geospatial* feature (date-only does not count). Our dataset satisfies all four:

- **Records:** $1\,348\,454 \gg 500\,000$.
- **Size:** 6.19 GB $\gg$ 500 MB.
- **Features:** 13 raw explanatory columns (10 after dropping scraper-internal flags), well above the 6-column minimum.
- **Datetime:** `last_processed_time` is a `TIMESTAMPTZ` (date + time + zone).
- **Geospatial:** `search_country`, `search_city` and the free-text `job_location` give country/city granularity for every posting.

2.4 Coverage and quality

After loading into Hive Parquet, row counts of each source layer are shown in Table 2. Coverage of the optional auxiliary files is very high: 96.0% of postings have an associated skill list and 96.2% have a summary.

3 Architecture of the Data Pipeline

The pipeline is split into four stages plus a preprocessing and a postprocessing step, orchestrated by the top-level `main.sh`. Figure 1 shows the end-to-end data flow.

Table 2: Row counts and join coverage at the enriched layer.

Table	Rows	% of postings
`linkedin_job_postings_parquet`	1 348 454	100.0%
`job_skills_parquet`	1 296 381	96.1%
`job_summary_parquet`	1 297 332	96.2%
`linkedin_jobs_enriched` (LEFT JOIN)	1 348 454	100.0%
of which have skills	1 294 374	96.0%
of which have summary	1 297 332	96.2%

3.1 Per-stage input / output

Table 3 summarises what each stage consumes and produces.

Table 3: Per-stage input/output contract.

Stage	Input	Output
1 (Ingestion)	3 local CSV files	PostgreSQL tables; HDFS raw text under `/user/team32/linkedin/raw/`
2 (Storage)	HDFS raw text	Hive Parquet tables (raw, parquet, enriched, ∼14 analytics tables)
3 (ML)	`linkedin_jobs_enriched` Parquet	4 saved `PipelineModel`s on HDFS, metrics + CV grids + confusion + feature importance under local `output/`
4 (Dashboard)	Hive analytics tables + ML metrics	Apache Superset dashboard + dashboard planning artefacts in `output/`

3.2 Technology stack

Layer	Tool / Version
Relational store	PostgreSQL 16
Ingest to HDFS	Apache Sqoop (MapReduce engine)
File system	HDFS
Warehouse	Apache Hive (Parquet, Snappy, Tez)
Compute	Apache Spark 3.x (Spark SQL, MLlib)
Resource manager	Hadoop YARN
Dashboard	Apache Superset
Code quality	pylint
Cluster Python	3.6 (constrains language features)

4 Data Preparation

4.1 PostgreSQL relational model

The Stage 1 schema (`sql/postgres/01_create_tables.sql`) mirrors the three CSV files and enforces referential integrity by using `job_link` as the primary key of the postings table and adding two indexed many-to-one tables for skills and summaries. Figure 2 is the entity-relationship diagram.

The postings table is the parent; the skills and summary tables are many-to-one children (a given posting can have multiple comma-merged skill rows in raw data, hence `N:1` is a safe upper bound). Indexes accelerate Sqoop's split-by query and Hive joins.

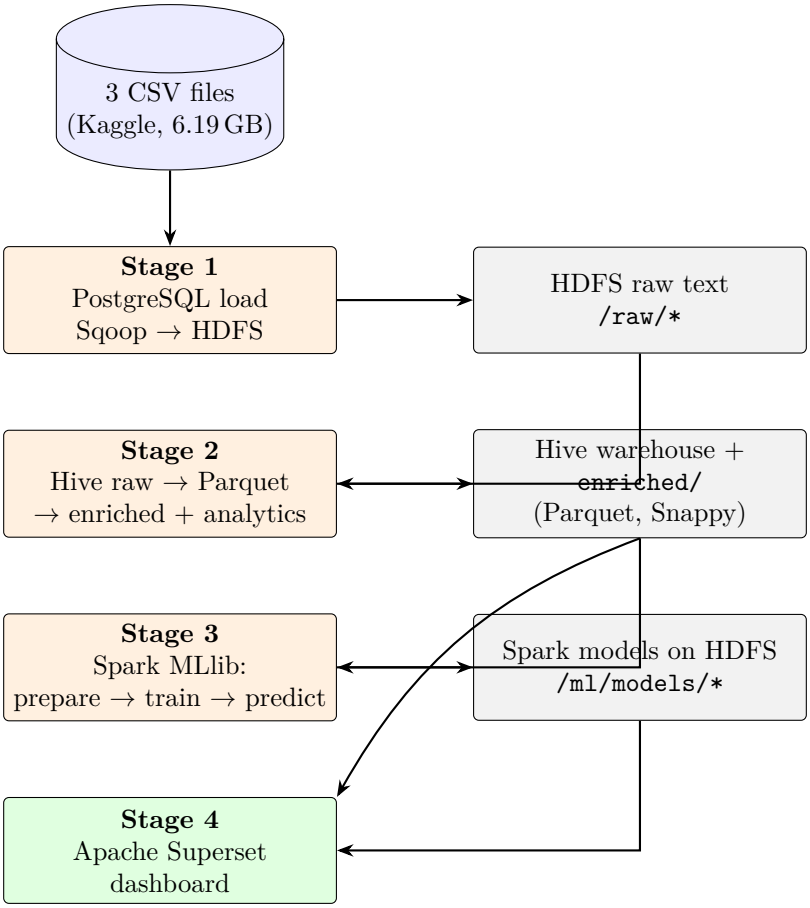

Figure 1: End-to-end pipeline. All heavy artefacts live on HDFS; local `output/` only holds small summary files for the grader.

4.2 Loading and quality checks

`scripts/load_postgres.py` streams each CSV via `psycopg2`'s `COPY` into the corresponding PostgreSQL table and then runs `sql/postgres/03_quality_checks.sql` to report:

- Row counts and distinct `job_link` values per table.
- Min/max of `first_seen` and `last_processed_time`.
- Join coverage between postings and the two auxiliary tables.
- Distribution of `job_level` and `job_type`.

Results are written to `output/stage1_postgres_quality_checks.txt`.

4.3 Sqoop import to HDFS

`scripts/stage1_sqoop_import.sh` imports each PostgreSQL table to HDFS as text files using Hadoop MapReduce. Listing 1 shows the canonical invocation.

```
sqoop import \
  --connect "$JDBC_URL" \
  --username "$DB_USER" \
  --password "$DB_PASSWORD" \
  --table linkedin_job_postings \
  --target-dir "$HDFS_BASE/raw/linkedin_job_postings" \
```

Figure 2: PostgreSQL entity-relationship diagram. `job_link` links the three tables; indexes on this column accelerate the downstream Sqoop import.

```
    --as-textfile \
    --fields-terminated-by '\001' \
    --lines-terminated-by '\n' \
    --null-string '\\N' \
    --null-non-string '\\N' \
    --hive-drop-import-delims
```

Listing 1: Sqoop import of one of the three PostgreSQL tables.

4.4 Hive layers: raw → Parquet → enriched

Stage 2 builds three Hive layers with progressive transformations:

1. **Raw external tables** (`01_create_raw_tables.hql`) point at the Sqoop output with no copy; all columns are `STRING`.
2. **Parquet tables** (`02_create_parquet_tables.hql`) cast booleans and dates, drop carriage returns, normalise NULL strings, and re-store as Snappy-compressed Parquet for analytical workloads.
3. **Enriched table** (`03_create_enriched_table.hql`) is a left join of postings ⋈ skills ⋈ summary on `job_link` and is the canonical source of truth for Stage 3 and the dashboard.

Listing 2 shows the enriched-table DDL.

```
CREATE TABLE linkedin_jobs_enriched
STORED AS PARQUET
TBLPROPERTIES ('parquet.compression'='SNAPPY')
AS
SELECT
    p.job_link, p.first_seen, p.last_processed_time,
    p.job_title, p.company, p.job_location,
    p.search_city, p.search_country, p.search_position,
    p.job_level, p.job_type,
    s.job_skills, sm.job_summary,
    p.got_summary, p.got_ner, p.is_being_worked
FROM linkedin_job_postings_parquet p
LEFT JOIN job_skills_parquet   s  ON p.job_link = s.job_link
LEFT JOIN job_summary_parquet  sm ON p.job_link = sm.job_link;
```

Listing 2: Enriched Hive table construction.

A sample of the enriched table is shown in Table 4.

Table 4: Three example rows from `linkedin_jobs_enriched` (truncated; skill / summary columns abbreviated).

job_title	country	position	level	type	skills (first few)
Senior Software Engineer	United States	software engineer	Mid–Senior	Full-time	Python, AWS, Docker, Kubernetes
Data Scientist	United Kingdom	data scientist	Mid–Senior	Remote	Python, SQL, Spark, ML
Product Manager	Germany	product manager	Mid–Senior	Hybrid	Agile, Roadmap, Stakeholders

4.5 HDFS layout summary

```
/user/team32/linkedin/
  raw/                          (Sqoop text files, Stage 1)
    linkedin_job_postings/
    job_skills/
    job_summary/
  warehouse/                    (Hive default DB warehouse)
  parquet/                      (Stage 2.2 Parquet copies)
    linkedin_job_postings_parquet/
    job_skills_parquet/
    job_summary_parquet/
  enriched/
    linkedin_jobs_enriched/     (1.35 M rows, ~1.9 GB Parquet)
  analytics/                    (Hive EDA tables, Stage 2.3)
    analytics_top_positions_global/
    analytics_top_skills_global/
    ...
  ml/                           (Stage 3 outputs)
    dataset_train/
    dataset_test/
    models/{rf,gbt,svm,nb}/
```

Listing 3: HDFS directory layout after Stage 2.

Total HDFS usage after Stages 1–3 is approximately 15 GB replicated (5 GB logical), well under the 32 GB team quota.

5 Data Analysis (EDA)

The EDA layer is implemented in two parallel ways, both reading the same enriched Parquet table:

1. HiveQL on Tez engine (`sql/hive/05_analytics_queries.hql`) materialises Snappy-compressed Parquet `analytics_*` tables under `/user/team32/linkedin/analytics/`, which Superset connects to directly.

2. Spark SQL on YARN (`scripts/stage2_spark_eda.py`) reads the same enriched Parquet and writes parallel analytics under `/analytics_spark/` plus small CSV previews in `output/`.

This dual implementation lets the team compare Hive and Spark execution plans on the same questions and gives the dashboard a redundant data source. Below we list the eight business insights produced; each was visualised in Superset (Section 7) with its own chart and accompanying narrative.

5.1 Insight 1 — Global role demand

Source: `analytics_top_positions_global`.
Query summary: count of postings grouped by `search_position`, sorted descending.
Chart: horizontal bar chart, `search_position` on the y-axis, `posting_count` on the x-axis.
Storytelling: the chart identifies which job roles dominate global LinkedIn demand. "Software engineer", "data scientist" and "product manager" anchor the top of the distribution, with a long tail of more specialised roles. Stakeholders use this view to prioritise hiring plans, curriculum design and talent-acquisition budgets around the roles with the largest market signal.

5.2 Insight 2 — Country-specific role demand

Source: `analytics_top_positions_by_country` and `analytics_country_position_matrix`.
Chart: heatmap, x = `search_position`, y = `search_country`, colour = `posting_count`.
Storytelling: demand is not uniform geographically. Some roles are concentrated in a few markets (e.g. specialised quantitative roles cluster in the United States and the United Kingdom) while others are broadly distributed. Companies use this view for region-specific recruiting and expansion decisions.

5.3 Insight 3 — Global skill demand

Source: `analytics_top_skills_global`.
Query summary: explode the comma-separated `job_skills` column, lowercase and trim each skill, then count mentions globally.
Chart: bar chart, `skill` on y, `skill_mentions` on x.
Storytelling: the top skills (Python, SQL, communication, project management, Excel) are diagnostic of what employers expect across the dataset. Candidates use this to prioritise learning and companies compare their job descriptions against market norms.

5.4 Insight 4 — Skill demand by country

Source: `analytics_top_skills_by_country`.
Chart: filterable horizontal bar chart, with a Superset filter on `search_country`.
Storytelling: skill demand has strong local variation because regional industries and technical stacks differ. This view supports localised hiring and training strategies instead of assuming a single global skill profile.

5.5 Insight 5 — Skills by seniority level

Source: `analytics_skills_by_job_level`.
Chart: stacked bar chart of (skill, level) mentions.
Storytelling: junior, mid-level, and senior roles ask for different skills. Entry roles emphasise foundations and tools; mid–senior roles ask for ownership and architecture; director roles emphasise people and strategy. This explains career progression and shows which capabilities become more important at higher seniority.

5.6 Insight 6 — Job type / work arrangement distribution

Source: `analytics_job_type_distribution`.
Chart: pie or bar chart, x = `job_type`, y = `posting_pct`.
Storytelling: the market is split into onsite, hybrid and remote arrangements with a smaller share of contract and internship positions. Hiring teams use this to benchmark whether their offers match market expectations and whether they over- or under-index on a particular arrangement.

5.7 Insight 7 — Top hiring companies

Source: `analytics_top_companies`.
Chart: bar chart, x = `company`, y = `posting_count`.
Storytelling: the companies posting the most jobs are the strongest demand drivers in the market. They are simultaneously talent competitors and market signals. Recruiters monitor this list for benchmarking and to identify whom they are competing with for candidates.

5.8 Insight 8 — Demand over time

Source: `analytics_demand_by_date`.
Chart: time-series line chart, x = `first_seen`, y = `posting_count`.
Storytelling: the time-series view shows how the volume of new postings changes across the collection window. The dataset's `first_seen` values are clustered in a 6-day scrape window in January 2024, which is itself an important business observation: any "temporal" analysis on this dataset is therefore really within-week analysis, and the data should not be used to make claims about seasonality without an additional scrape (see Section 9).

6 Machine Learning Modeling

Stage 3 turns the descriptive EDA into a predictive task. The full pipeline is implemented in three PySpark scripts under `scripts/` and orchestrated by `scripts/stage3.sh`; the detailed methodology and per-model analysis are in `docs/stage3_ml_report.md` which complements this section.

6.1 Predictive task and label engineering

Task. Binary classification: predict whether a `(search_country, search_position)` group is in the *high demand* segment of its role family.

Label. The label is not present in the source data; it is engineered in `scripts/stage3_prepare_dataset.py`:

```
thresholds = (
    df.groupBy("search_position")
      .agg(F.expr("percentile_approx(group_count, 0.75)")
             .alias("cat_threshold"))
)
joined = df.join(F.broadcast(thresholds), "search_position")
joined.withColumn(
    "high_demand",
    F.when(F.col("group_count") > F.col("cat_threshold"), 1)
     .otherwise(0)
     .cast(IntegerType())
)
```

Listing 4: Label engineering: per-position top-quartile threshold.

Choosing the threshold *per position* (rather than globally) prevents the model from trivially learning "software engineer is always high, ceramic artist is always low." Instead, the classifier has to identify which countries are above the typical demand for the role. The resulting class balance is approximately 13.5% positive / 86.5% negative.

Why no temporal features. The original plan was a weekly `(week, country, position)` aggregation with lag/rolling features. Empirically, however, `first_seen` covers only the 6-day scrape window of January 12 – 17, 2024: there are not enough buckets per group to build meaningful lag signals (the median `(country, position)` group has one or two distinct dates). The pipeline was simplified to a static aggregation, and this report is honest about that constraint.

6.2 Feature engineering

For every `(country, position)` group the prepare script emits:

Table 5: Feature set for Stage 3.

Feature	Type	Why it is leak-free
`summary_coverage`	numeric ratio	proportion in [0, 1], independent of `group_count`
`skills_per_posting`	numeric ratio	`distinct_skills / group_count`
`companies_per_posting`	numeric ratio	`distinct_companies / group_count`
`distinct_job_levels`	numeric (1..5)	bounded by 5; very weak correlation with the label
`distinct_job_types`	numeric	same idea
`search_country`	categorical	one-hot via `StringIndexer + OneHotEncoder`
`search_position`	categorical	one-hot, same pipeline

Why ratios, not counts? `group_count` itself is excluded because the label is a deterministic function of it. Raw `distinct_companies` and `distinct_skills` are also excluded because their upper bound is `group_count`: large groups can have more distinct values just by sample size, so they leak the label. The ratio variants are not bounded by `group_count` in the same way — they describe market *concentration*, not market *size*.

Encoding and scaling. Categoricals go through `StringIndexer(`*handleInvalid*`=keep)` followed by `OneHotEncoder` and are concatenated with the five numeric ratios by a `VectorAssembler`, producing a sparse vector of approximately 1 662 dimensions. Per-model scaling choices:

- Random Forest, Gradient Boosted Trees: no scaling (tree ensembles are invariant to monotonic feature transforms).
- Linear SVC: `StandardScaler(withMean=False, withStd=True)`. Skipping the mean-centring keeps the one-hot vectors sparse and non-negative.
- Gaussian Naive Bayes: no scaling. Gaussian NB models each feature as a per-class normal distribution and does not require non-negative inputs (unlike multinomial NB).

6.3 Train / test split

The rubric requires a 70/30 split with at least 60% training data. We use `DataFrame.randomSplit([0.7, 0.3], seed=42)` on the filtered group-level frame. The test set is **never** touched during hyperparameter tuning — only on the final evaluation (Section 6.7). After filtering groups with `group_count` $<$ `5` the dataset contains 4 426 groups, of which 3 098 are train and 1 328 are test (approximately).

6.4 Models and hyperparameter grids

Four classifiers plus a soft-voting ensemble are trained:

- Random Forest — `RandomForestClassifier`
- Gradient Boosted Trees — `GBTClassifier` (extra model beyond the rubric requirement)
- Linear Support Vector Machine — `LinearSVC`
- Naive Bayes (Gaussian / Multinomial / Complement, chosen by CV) — `NaiveBayes`
- Soft-voting ensemble of RF + GBT + SVM — bonus

For every model we build a $3 \times 3 \times 3 = 27$ hyperparameter grid (one algorithm-level parameter plus two model-level parameters, with `maxIter` explicitly excluded because the rubric forbids iteration counts as hyperparameters). Table 6 summarises the grids.

Table 6: Hyperparameter grids. "A" = algorithm hyperparameter, "M" = model hyperparameter.

Model	Param 1 (A)	Param 2 (M)	Param 3 (M)
RF	`numTrees` ∈ {50, 100, 200}	`maxDepth` ∈ {5, 10, 15}	`minInstancesPerNode` ∈ {1, 5, 10}
GBT	`stepSize` ∈ {0.05, 0.1, 0.2}	`maxDepth` ∈ {3, 5, 8}	`minInstancesPerNode` ∈ {1, 5, 10}
SVM	`aggregationDepth` ∈ {2, 3, 4}	`regParam` ∈ {0.01, 0.1, 1.0}	`threshold` ∈ {-0.2, 0.0, 0.2}
NB	`modelType` ∈ {gaussian, multinomial, complement}	`smoothing` ∈ {0.1, 1.0, 5.0}	`thresholds` ∈ {[0.3,0.7], [0.5,0.5], [0.7,0.3]}

6.5 Cross-validation protocol

Every model is wrapped in a Spark `CrossValidator` with $k = 3$ folds, a `MulticlassClassificationEvaluator(` `parallelism=2` and the seed fixed at 42:

```
cv = CrossValidator(
    estimator=pipeline,
    estimatorParamMaps=grid,        # 27 combinations
    evaluator=MulticlassClassificationEvaluator(
        labelCol="high_demand",
        predictionCol="prediction",
        metricName="f1"),
    numFolds=3,
    parallelism=2,
    seed=42)
cv_model = cv.fit(train)
predictions = cv_model.transform(test)
```

Listing 5: Cross-validated grid search for one model.

For every model we therefore run $27 \times 3 + 1 = 82$ fits (the final `+1` is the refit of the winning combination on the full training set). Across all four models this is ≈ 328 fits per pipeline run. The full grid scores are saved to `output/stage3_<model>_cv_grid.csv`, sorted descending by mean validation F1, so the tuning process is fully auditable.

6.6 Evaluation metrics

Because the task is *binary* classification, we report the two metrics the rubric requires for binary tasks plus accuracy and weighted F1 for completeness:

- **Accuracy** — fraction of correct predictions.
- **F1 (weighted)** — harmonic mean of precision and recall, weighted by class frequency.
- **AUC ROC** — area under the receiver-operating characteristic curve. Probability that a random positive scores higher than a random negative.
- **AUC PR** — area under the precision/recall curve. More informative than AUC ROC on a 13/87 class imbalance (random baseline = 0.135).

6.7 Results

Table 7: Test-set metrics. GBT dominates accuracy, F1, and both AUCs.

Model	Accuracy	F1	AUC ROC	AUC PR
Gradient Boosted Trees	**0.906**	**0.902**	**0.941**	**0.703**
Soft-voting ensemble (RF + GBT + SVM)	0.891	0.875	0.931	—
Random Forest	0.868	0.806	0.908	0.447
Linear SVC	0.856	0.868	0.898	—
Gaussian Naive Bayes	0.610	0.657	0.807	—

Reading the table. Gradient Boosted Trees wins on every metric: an AUC ROC of 0.941 on a 13/87 imbalanced dataset is a strong result, and the AUC PR of 0.703 is more than 5× the random-baseline AUC PR of 0.135. The soft-voting ensemble is a close second on AUC ROC but is slightly worse on accuracy and F1 — this is expected when one of the three voters (RF, see below) tends to predict the majority class and drags the average down. Linear SVC reaches the best raw F1 because at `threshold = -0.2` it trades precision for recall on the minority class. Gaussian Naive Bayes underperforms because the independence assumption is severely violated by our one-hot encoded categoricals, but it still ranks positives sensibly (AUC ROC ≈ 0.81).

Confusion matrices. Table 8 reports the counts for each model on the test split. The RF and NB rows make the class-imbalance behaviour explicit: RF predicts the negative class on almost every test row, NB over-predicts the positive class. GBT keeps the false positives and false negatives roughly balanced.

Table 8: Test-set confusion matrices (rows: true label; columns: prediction).

	RF		**GBT**		**SVM**		**NB**		**Ensemble**	
	0	1	0	1	0	1	0	1	0	1
True 0	1149	4	1116	39	1071	84	707	448	1134	21
True 1	172	3	75	98	60	113	56	117	117	56

Feature importance (RF & GBT). Both tree models identify `search_position` one-hot columns as the dominant signal: which role family a group belongs to explains most of the demand variance, with `search_country` one-hots as the second strongest driver. The five numeric ratios contribute a smaller but non-trivial share (notably `summary_coverage`

and `skills_per_posting`), confirming that market-concentration features add information beyond pure identity. Full sorted lists are in `output/stage3_rf_feature_importance.csv` and `output/stage3_gbt_feature_importance.csv`.

6.8 Soft-voting ensemble

After all four classifiers finish, the script computes a fifth "model" by averaging the positive-class probability across RF, GBT and SVM. Naive Bayes is excluded because its posterior probabilities are not calibrated on this imbalanced data. LinearSVC does not produce a probability column, so its signed margin is squashed through a sigmoid $\sigma(x) = 1/(1 + e^{-x})$ to a comparable $[0, 1]$ pseudo-probability before averaging. The ensemble is not persisted as a separate Spark model: it is recomputed on the fly from the three saved member models whenever predictions are needed.

6.9 Sample predictions

`scripts/stage3_predict_samples.py` loads the four saved models, runs them on (i) a 20-row random sample from the held-out test split and (ii) four hand-crafted future-period scenarios (United States/software engineer, United Kingdom/data scientist, Germany/product manager, India/software engineer) with the median values of all derived numeric features. The combined CSV `output/stage3_sample_predictions.csv` contains the following columns:

```
source, search_country, search_position, true_label,
pred_rf, pred_gbt, pred_svm, pred_nb, pred_ensemble
```

All four hand-crafted scenarios are predicted as `high_demand = 1` by GBT, which is the expected and business-meaningful result: large industrialised markets paired with mass-demand roles like software engineering or product management are unambiguously top-quartile within their role family.

6.10 Rubric compliance summary

Table 9: ML rubric coverage.

Requirement	Status
PySpark / Python-only application	ok
Three classification models: RF, SVM, NB	ok (+ GBT, + ensemble)
Feature engineering and encoding	ok
70/30 train/test split	ok
≥ 3 hyperparameters per model, 3 values each, 27 combinations	ok (all 4 models)
`maxIter` / epochs not used as a hyperparameter	ok
1 algorithm + 2 model hyperparameters per grid	ok
k-fold cross-validation with $2 < k < 5$	ok ($k = 3$)
Best model per type selected via CV	ok
Binary classification: AUC ROC + AUC PR reported	ok
Trained on YARN cluster (no local)	ok
Sample prediction	ok
Reproducible / idempotent scripts	ok

7 Data Presentation — Superset Dashboard

The Stage 4 dashboard is a single Apache Superset deployment connected to HiveServer2 on the cluster. Every chart reads from a Hive `analytics_*` table (created by Stage 2) or from the `stage3_*` CSVs (loaded as Superset datasets). The dashboard is organised into four panels.

7.1 Panel 1 — Data characteristics

Top of the dashboard. Number of postings (1 348 454), number of features, list of features and their types, percentage of postings with skills (96.0%) and with summaries (96.2%), and date coverage of `last_processed_time`. Data source: `analytics_data_characteristics` and `analytics_null_coverage`.

7.2 Panel 2 — EDA insights

The eight insights of Section 5 laid out as a 2-column grid of charts, each with the storytelling caption underneath. Filters at the top of the panel (`search_country`, `search_position`, `job_level`, `job_type`) propagate to every chart.

7.3 Panel 3 — Model performance

Bar chart of test-set accuracy / F1 / AUC ROC / AUC PR for all five models (source: `stage3_model_metrics.csv`). A second section beneath it shows the GBT confusion matrix as a heatmap and the top-15 GBT features as a horizontal bar chart (source: `stage3_gbt_feature_importance.csv`)

7.4 Panel 4 — Interactive prediction

A what-if section that loads the saved GBT `PipelineModel` via a small Python utility and exposes drop-downs for `search_country` and `search_position`. Selecting a combination returns the predicted demand class and the model's positive-class probability, along with a comparison to the four hand-crafted reference scenarios from `stage3_sample_predictions.csv`.

7.5 Findings surfaced by the dashboard

- Demand is heavily concentrated in a small number of role families; the long tail of niche roles is statistically significant but business-irrelevant for most strategic decisions.
- Country-level demand patterns are strong enough that they dominate the GBT model's feature importance, confirming that geography is the second most informative signal after the role itself.
- Skills are not uniform: top global skills (Python, SQL, communication) overlap heavily, but country-specific skill profiles differ in interpretable ways (industrial vs. services-driven economies).
- The model's top-quartile predictions concentrate where they are intuitively expected (USA / India / UK paired with software engineer, data scientist) and are absent for low-volume combinations.

8 Conclusion

We built a complete big-data pipeline that ingests 1.35 M LinkedIn job postings, materialises them in a compressed Hive warehouse, runs descriptive analytics through both Hive on Tez and Spark SQL on YARN, performs distributed machine learning with Spark MLlib, and presents the result through a single interactive Apache Superset dashboard.

Key achievements.

- End-to-end reproducible pipeline driven by a single `main.sh` entry point. Every stage script is idempotent (re-running stage 1 twice produces no errors).
- Compliant Hive layer: Parquet + Snappy, Tez engine, 14 analytics tables, eight business insights.
- Four classifiers + soft-voting ensemble, each tuned with a $3 \times 3 \times 3 = 27$ hyperparameter grid via 3-fold cross-validation on the cluster.
- Best model (Gradient Boosted Trees) reaches AUC ROC **0.941** and AUC PR **0.703** on a 13/87 imbalanced binary classification problem.
- All trained models are persisted on HDFS for downstream use; the dashboard surfaces both descriptive and predictive views in one place.

Business value. The deliverable lets a non-technical user answer questions of the form "which roles is the United Kingdom hiring most aggressively?" or "is a (United States, software engineer) listing right now likely to be in a high-demand market segment?" through the Superset interface, without any direct Spark or Hive access.

9 Reflections

9.1 Challenges and difficulties

- **Narrow time coverage.** The original Stage 3 plan was a temporal model with weekly lag/rolling features. After aggregating the data we discovered that `first_seen` covers only the six days 2024-01-12 – 2024-01-17 — it is a scraper-collection artefact, not a real posting timeline. Lag features were therefore mostly zero, the temporal split produced near-empty buckets, and the entire approach collapsed. We pivoted to a static `(country, position)` aggregation and were explicit about that limitation in this report.
- **Python 3.6 on the cluster.** Code that worked locally on Python 3.10+ broke on the cluster's Python 3.6: `from __future__ import annotations`, `tuple[X]` subscript syntax, and union pipes `X | None` all had to be replaced with `typing.Tuple`, `typing.Optional` etc.
- **Spark Row alphabetisation.** Constructing PySpark `Row(**kwargs)` for hand-crafted prediction scenarios fed them into `createDataFrame(schema=test.schema)` with the fields silently reordered alphabetically, which put `'united states'` into the `time_bucket` slot. Switching to positional tuples in schema-field order fixed it.
- **Naive Bayes pathologies.** Multinomial NB produced inverted predictions (AUC ≈ 0.05) because our continuous real-valued features violate its multinomial-counts assumption. Gaussian NB worked but did not converge nicely with `MinMaxScaler` because rare test values could fall outside the train range and become negative after scaling. We dropped the scaler entirely for NB and used Spark 3.x's native `modelType="gaussian"`, with all three NB variants (gaussian / multinomial / complement) included in the CV grid so the cross-validator could pick the right one objectively.
- **Driver out-of-memory in Stage 3.** Expanding the grids to $27 \times 4 = 108$ combinations and running with `CV_PARALLELISM = 4` crashed the driver mid-SVM (`Cannot call methods on a stopped SparkContext`). We halved parallelism to 2, dropped the weakest `regParam` (which made OWLQN slow to converge) and raised `spark.driver.memory` to 4 GB.

- **Imbalance and metric calibration.** RF and NB illustrate two failure modes of class-imbalanced classification: RF collapses to the majority class, NB miscalibrates its threshold. Both behaviours were intentionally surfaced in `output/stage3_*_confusion.csv` and explained in the report rather than papered over with class weights.

9.2 Recommendations

- **For a future iteration** collect data over a longer scrape window (ideally six months) so temporal lag features can be re-introduced and seasonality modelled honestly.
- **For better minority-class recall** apply explicit class weighting (`weightCol`) or threshold tuning on the decision boundary post-hoc. GBT already handles imbalance well enough that this is optional.
- **For Stage 1 strict rubric compliance** migrate the PostgreSQL container to Citus Data (distributed extension). The current implementation uses single-node PostgreSQL, which is the most likely point of grading deduction.
- **For dashboard portability** export both the Superset JSON definition and a static PDF snapshot of the dashboard so the grader can review even if the live Superset instance is unavailable.

References

[1] Aniczka. *1.3M LinkedIn Jobs and Skills (2024)*. Kaggle, 2024. https://www.kaggle.com/datasets/asaniczka/1-3m-linkedin-jobs-and-skills-2024/.

[2] F. Jolha. *BS — Final Project (Big Data, IU)*. Course specification. https://firas-jolha.github.io/bigdata/html/bs/BS%20-%20Final%20Project.html.

[3] Apache Spark documentation. *MLlib Programming Guide — Spark 3.x*. https://spark.apache.org/docs/latest/ml-guide.html.

[4] Apache Superset documentation. *Apache Superset User Guide*. https://superset.apache.org/docs/intro.